GNU nano 8.7.1

```c
#include <stdio.h>
#define ROWS 5
#define COLS 8
// Function prototypes
void display_seats(int seats[ROWS][COLS]);
int book_one_seat(int seats[ROWS][COLS], int row, int col);
int cancel_booking(int seats[ROWS][COLS], int row, int col);
void occupancy_report(int seats[ROWS][COLS]);
int group_reservation(int seats[ROWS][COLS], int num_seats);
int main() {
    // 5 rows and 8 columns, initialized to 0 (Available)
    int seats[ROWS][COLS] = {0};
    int choice, row, col, num_seats, result;
    do {
        printf("\n--- Menu Options ---\n");
        printf("1. Display seats\n");
        printf("2. Book one seat\n");
        printf("3. Cancel booking\n");
        printf("4. Occupancy report\n");
        printf("5. Group reservation\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            printf("Invalid input. Please enter a number.\n");
            while (getchar() != '\n'); // clear buffer
            continue;
        }
        switch (choice) {
            case 1:
                display_seats(seats);
                break;
            case 2:
                printf("Enter row (1-5) and seat (1-8): ");
                scanf("%d %d", &row, &col);
                result = book_one_seat(seats, row, col);
                if (result == 1)
                    printf("Seat booked successfully.\n");
                else
                    printf("Booking failed. Seat is either already booked or out of range.\n");
                break;
            case 3:
                printf("Enter row (1-5) and seat (1-8) to cancel: ");
                scanf("%d %d", &row, &col);
                result = cancel_booking(seats, row, col);
                if (result == 1)
                    printf("Booking cancelled successfully.\n");
                else
                    printf("Cancellation failed. Seat is either already free or out of range.\n");
                break;
            case 4:
                occupancy_report(seats);
                break;
            case 5:
                printf("Enter number of adjacent seats to book: ");
                scanf("%d", &num_seats);
                result = group_reservation(seats, num_seats);
                if (result == 1)
                    printf("Group reservation successful.\n");
                else
                    printf("Group reservation failed. Not enough consecutive seats in any row.\n");
                break;
            case 6:
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mar
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy

Search

GNU nano 8.7.1

```
            case 6:
                printf("Display a closing message and terminate the program.\n");
                break;
            default:
                printf("Invalid choice. Please try again.\n");
        }
    } while (choice != 6);
    return 0;
}
// 1. Display all 40 array elements to 0/1 legend
void display_seats(int seats[ROWS][COLS]) {
    printf("\nAuditorium Seating Arrangement (0: Available, 1: Booked):\n");
    printf("    1 2 3 4 5 6 7 8\n");
    for (int i = 0; i < ROWS; i++) {
        printf("R%d ", i + 1);
        for (int j = 0; j < COLS; j++) {
            printf("%d ", seats[i][j]);
        }
        printf("\n");
    }
}
// 2. Accept 1-based row/seat numbers. Return 1 if booking succeeds, otherwise return 0.
int book_one_seat(int seats[ROWS][COLS], int row, int col) {
    int r = row - 1;
    int c = col - 1;
    if (r < 0 || r >= ROWS || c < 0 || c >= COLS) return 0;
    if (seats[r][c] == 0) {
        seats[r][c] = 1;
        return 1;
    }
    return 0;
}
// 3. Accept 1-based row/seat numbers. Return 1 if cancellation succeeds, otherwise return 0.
int cancel_booking(int seats[ROWS][COLS], int row, int col) {
    int r = row - 1;
    int c = col - 1;
    if (r < 0 || r >= ROWS || c < 0 || c >= COLS) return 0;
    if (seats[r][c] == 1) {
        seats[r][c] = 0;
        return 1;
    }
    return 0;
}
// 4. Show booked seats, available seats, occupancy percentage, and busiest row(s).
void occupancy_report(int seats[ROWS][COLS]) {
    int total_booked = 0;
    int row_booked[ROWS] = {0};

    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            if (seats[i][j] == 1) {
                total_booked++;
                row_booked[i]++;
            }
        }
    }
    int total_seats = ROWS * COLS;
    int total_available = total_seats - total_booked;
    float percentage = ((float)total_booked / total_seats) * 100.0f;
    // Find max booked in a row
    int max_booked = -1;
    for (int i = 0; i < ROWS; i++) {
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo

Search

yokshita@yokshithachowdary

GNU nano 8.7.1

```
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            if (seats[i][j] == 1) {
                total_booked++;
                row_booked[i]++;
            }
        }
    }
    int total_seats = ROWS * COLS;
    int total_available = total_seats - total_booked;
    float percentage = ((float)total_booked / total_seats) * 100.0f;
    // Find max booked in a row
    int max_booked = -1;
    for (int i = 0; i < ROWS; i++) {
        if (row_booked[i] > max_booked) {
            max_booked = row_booked[i];
        }
    }
    printf("\n--- Occupancy Report ---\n");
    printf("Booked seats: %d\n", total_booked);
    printf("Available seats: %d\n", total_available);
    printf("Occupancy percentage: %.2f%%\n", percentage);
    printf("Busiest row(s): ");
    if (max_booked == 0) {
        printf("None (no seats booked yet)\n");
    } else {
        for (int i = 0; i < ROWS; i++) {
            if (row_booked[i] == max_booked) {
                printf("Row %d (%d seats) ", i + 1, max_booked);
            }
        }
        printf("\n");
    }
}
// 5. Accept group size 2-8. Return 1 if booked or 0 if rejected. Display allocated row and first seat on success.
int group_reservation(int seats[ROWS][COLS], int num_seats) {
    if (num_seats < 2 || num_seats > 8) return 0;

    for (int i = 0; i < ROWS; i++) {
        int consecutive = 0;
        int start_col = -1;
        for (int j = 0; j < COLS; j++) {
            if (seats[i][j] == 0) {
                if (consecutive == 0) start_col = j;
                consecutive++;
                if (consecutive == num_seats) {
                    // Book the consecutive seats
                    for (int k = start_col; k < start_col + num_seats; k++) {
                        seats[i][k] = 1;
                    }
                    printf("Allocated Row %d, starting at seat %d\n", i + 1, start_col + 1);
                    return 1;
                }
            } else {
                consecutive = 0;
                start_col = -1;
            }
        }
    }
    return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy

Search

yokshita@yokshithachowdary

```
Auditorium Seating Arrangement (0: Available, 1: Booked):
   1 2 3 4 5 6 7 8
R1 0 0 0 0 0 0 0 0
R2 0 0 0 0 0 0 0 0
R3 0 0 0 0 0 0 0 0
R4 0 0 0 0 0 0 0 0
R5 0 0 0 0 0 0 0 0

--- Menu Options ---
1. Display seats
2. Book one seat
3. Cancel booking
4. Occupancy report
5. Group reservation
6. Exit
Enter your choice: 2
Enter row (1-5) and seat (1-8): 2 3
Seat booked successfully.

--- Menu Options ---
1. Display seats
2. Book one seat
3. Cancel booking
4. Occupancy report
5. Group reservation
6. Exit
Enter your choice: 3
Enter row (1-5) and seat (1-8) to cancel: 2 3
Booking cancelled successfully.

--- Menu Options ---
1. Display seats
2. Book one seat
3. Cancel booking
4. Occupancy report
5. Group reservation
6. Exit
Enter your choice: 4

--- Occupancy Report ---
Booked seats: 0
Available seats: 40
Occupancy percentage: 0.00%
Busiest row(s): None (no seats booked yet)

--- Menu Options ---
1. Display seats
2. Book one seat
3. Cancel booking
4. Occupancy report
5. Group reservation
6. Exit
Enter your choice: 5
Enter number of adjacent seats to book: 3
Allocated Row 1, starting at seat 1
Group reservation successful.

--- Menu Options ---
1. Display seats
2. Book one seat
3. Cancel booking
4. Occupancy report
5. Group reservation
6. Exit
Enter your choice: 6
Display a closing message and terminate the program.
```

Search